

[CS230] Optimization Algorithms: Momentum & RMSprop

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-4. Neural Networks Basics - *Completed*

Chapter 5. Practical Aspects of Deep Learning (Current Unit)

- 5.1-5.5 Regularization & Data Setup - *Completed*
- 5.6 Mini-batch Gradient Descent - *Completed*
- **5.7 Optimization Algorithms (Momentum & RMSprop)**
 - * Exponential Weighted Moving Average (The Trend)
 - * Momentum (Physics: Velocity)
 - * RMSprop (Adaptive Learning Rate)
 - * Implementation
- 5.8 Adam Optimizer - *Upcoming*

지난 시간 복습 및 연결

지난 시간에 우리는 데이터를 작은 덩어리로 쪼개 학습하는 **미니 배치 경사 하강법**을 배웠습니다. 속도는 빨라졌지만, 그래프를 보면 여전히 최적점을 향해 곧바로 가지 못하고 **지그재그(Zigzag)**로 **진동(Oscillation)**하며 내려갑니다. 이 진동을 줄이고, 최적해를 향해 '**가속도(Acceleration)**'를 붙일 수는 없을까요? 물리학의 관성을 이용한 **Momentum**과, 보폭을 자동으로 조절하는 **RMSprop**이 그 해답입니다.

1 Unit Overview

□ 핵심 목표

이 단원은 단순한 경사 하강법을 넘어선 **고급 최적화 알고리즘** 두 가지를 다룹니다.

- **기초:** 시계열 데이터의 트렌드를 추출하는 **지수 가중 이동 평균(EWMA)**을 이해합니다.
- **Momentum:** 과거의 기울기를 누적하여 관성을 만드는 원리를 배웁니다.
- **RMSprop:** 기울기의 크기(제곱)에 따라 학습 보폭을 조절하는 적응형 알고리즘을 익힙니다.
- **구현:** 두 알고리즘의 수식을 Python 코드로 옮기고 하이퍼파라미터(β)를 설정합니다.

2 Essential Terminology

용어	기호	설명
EWMA	v_t	지수 가중 이동 평균. 최근 데이터의 경향성을 나타냄.
Momentum	v	관성(속도). 과거의 진행 방향을 유지하려는 성질.
RMSprop	S	Root Mean Square Prop. 기울기 제곱을 이용해 보폭 조절.
Beta	β	과거 데이터를 얼마나 기억할지 결정하는 계수 (0.9 등).

3 Core Concepts: 가속의 원리

3.1 1. 지수 가중 이동 평균 (Exponentially Weighted Moving Average)

이 알고리즘들의 기초가 되는 수학입니다.

$$v_t = \beta v_{t-1} + (1 - \beta)\theta_t$$

- $\beta = 0.9$: 최근 10일간의 평균 ($\frac{1}{1-0.9} = 10$)
- $\beta = 0.98$: 최근 50일간의 평균 ($\frac{1}{1-0.98} = 50$)
- β 가 클수록 그래프가 부드러워지지만(Smoothing), 변화에 둔감해집니다(Latency).

3.2 2. Momentum (관성)

”공이 언덕을 굴러 내려갈 때 속도가 붙는 물리 법칙”

□ 얼음판 위의 쇠구슬 (직관적 비유)

- **SGD (일반 경사 하강법)**: 마찰력이 무한대인 바닥. 힘(기울기)을 주면 움직이고, 안 주면 딱 멈춥니다. 방향이 바뀌면 즉시 꺾입니다 (지그재그).
- **Momentum**: 마찰력이 없는 얼음판. 힘을 주지 않아도 기존에 내려오던 속도(Velocity, v) 때문에 계속 미끄러져 내려갑니다. 이 관성이 진동을 상쇄하고 웅덩이(Local Minima)를 넘게 해줍니다.

□ Update Rule (수학적 원리)

1. 속도 계산: $v = \beta v + (1 - \beta)dW$
2. 파라미터 업데이트: $W = W - \alpha v$
(α : 학습률, β : 보통 0.9 사용)

3.3 3. RMSprop (Root Mean Square Propagation)

”가파른 곳은 천천히, 완만한 곳은 빠르게”

제프리 힌튼 교수가 제안한 방법입니다. 기울기(dW)의 크기를 보고 보폭을 조절합니다.

- 원리: 학습률 α 를 $\sqrt{S}$ 로 나눠줍니다.
- 효과:

- 기울기가 큼(S 큼) $\rightarrow$ 분모가 커짐 $\rightarrow$ 업데이트 폭 감소 (진동 억제)
- 기울기가 작음(S 작음) $\rightarrow$ 분모가 작아짐 $\rightarrow$ 업데이트 폭 증가 (가속)

□ Update Rule (수학적 원리)

1. 제곱 평균: $S = \beta_2 S + (1 - \beta_2) dW^2$ (요소별 제곱)
2. 파라미터 업데이트: $W = W - \alpha \frac{dW}{\sqrt{S + \epsilon}}$
(ϵ : 0으로 나누기 방지용, 10^{-8})

4 Implementation: Momentum & RMSprop

```

1 import numpy as np
2
3 def update_with_momentum(parameters, grads, v, beta, learning_rate):
4     """
5     v: 속도(Velocity) 딕셔너리 초기값은( 0)
6     beta: Momentum 계수 보통( 0.9)
7     """
8     L = len(parameters) // 2
9
10    for l in range(1, L + 1):
11        # 1. 속도(v) 업데이트 관성(누적)
12        v["dW" + str(l)] = beta * v["dW" + str(l)] + (1 - beta) * grads["dW" + str(l)]
13        v["db" + str(l)] = beta * v["db" + str(l)] + (1 - beta) * grads["db" + str(l)]
14
15        # 2. 파라미터 업데이트 (를v 빼줌)
16        parameters["W" + str(l)] -= learning_rate * v["dW" + str(l)]
17        parameters["b" + str(l)] -= learning_rate * v["db" + str(l)]
18
19    return parameters, v
20
21 def update_with_rmsprop(parameters, grads, s, beta2, learning_rate, epsilon=1e-8):
22     """
23     s: 제곱 평균(Squared Gradient) 딕셔너리
24     beta2: RMSprop 계수 보통( 0.999)
25     """
26     L = len(parameters) // 2
27
28    for l in range(1, L + 1):
29        # 1. 제곱 평균(s) 업데이트 기울기(제곱 주의!)
30        s["dW" + str(l)] = beta2 * s["dW" + str(l)] + (1 - beta2) * np.square(grads["dW" + str(l)])
31        s["db" + str(l)] = beta2 * s["db" + str(l)] + (1 - beta2) * np.square(grads["db" + str(l)])
32
33        # 2. 파라미터 업데이트 적응형(학습률)
34        # 분모에 sqrt(s) + epsilon
35        parameters["W" + str(l)] -= learning_rate * (grads["dW" + str(l)] / (np.sqrt(s["dW" + str(l)] + epsilon)))
36        parameters["b" + str(l)] -= learning_rate * (grads["db" + str(l)] / (np.sqrt(s["db" + str(l)] + epsilon)))
37
38    return parameters, s

```

Listing 1: Optimization Algorithms Implementation

5 FAQ & Pitfalls

변수 초기화 실수 (오해 방지 가이드)

v 와 s 는 학습 루프(Iteration)가 돌 때마다 초기화하면 안 됩니다! 그러면 관성이 사라집니다. 반드시 학습 시작 전(Epoch 0 이전)에 한 번만 0으로 초기화하고, 계속 값을 누적해야 합니다.

Q. β (Momentum)와 β_2 (RMSprop) 값은 튜닝해야 하나요?

A. 보통은 기본값($\beta = 0.9, \beta_2 = 0.999$)을 그대로 씁니다. 이 값들이 경험적으로 대부분의 문제에 잘 작동합니다. 학습률(α) 튜닝이 훨씬 중요합니다.

다음 단계 (Next Step)

우리는 최고의 가속 엔진 두 개를 얻었습니다.

- **Momentum:** 관성을 이용하여 속도를 높임.
- **RMSprop:** 보폭을 조절하여 진동을 줄임.

”둘 다 쓰면 안 되나요?” 당연히 됩니다. 이 둘을 결합한 것이 바로 **Adam (Adaptive Moment Estimation)**입니다. 현재 딥러닝 세계를 지배하고 있는 Adam 알고리즘을 다음 시간에 완성하고, 최적화 단원을 마무리하겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **Momentum:** $v \leftarrow dW$. 과거의 속도를 유지하여 Local Minima 탈출 및 가속.
2. **RMSprop:** $S \leftarrow dW^2$. 기울기가 크면 학습률을 낮춰 진동을 방지.
3. **Math:** dW^2 은 요소별 제곱이다. 나눗셈 시 ϵ 을 더해 에러를 방지한다.
4. **Hyperparam:** $\beta = 0.9, \beta_2 = 0.999$ 가 국룰(Standard)이다.